



AUDIT REPORT

February 2026

For



Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	08
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 Informational Issues	14
1. Use of Floating Pragma	14
Functional Tests	15
Automated Tests	15
Threat Model	16
Closing Summary & Disclaimer	25



Executive Summary

Project Name	TRIGGERMINT AI CAPITAL
Protocol Type	Custodial ERC20
Project URL	https://triggermint-ai-capital.com/
Overview	TMREMinimalCustody is a fixed-supply ERC20 token implementing whitelist-gated transfers, an admin-controlled trading enable switch, and a custodial settlement model where only a designated Transfer Agent can initiate transfers (optionally disabled), with manual off-chain redemption supported via agent-authorized burning.
Audit Scope	The scope of this Audit was to analyze the TRIGGERMINT AI CAPITAL Smart Contracts for quality, security, and correctness.
Source Code link	https://etherscan.io/address/0x7Cb940b9ee43922caD6dBc52A2D864991FBFb1f5
Contracts in Scope	TRIGGERMINT_AI_CAPITAL_REAL_ESTATE.sol
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	18th February 2026
Updated Code Received	19th February 2026
Review 2	19th February 2026
Fixed In	https://etherscan.io/address/0xE582d9aea588C3598cC9CBe80e0D0059A0119b7f#code



Note

The QuillAudits Team compiled the contract using Foundry and verified that the deployed bytecode matches the code under audit. The match is exact. Please find below the compilation details.

Compilation Details:

Solidity version: 0.8.20

Optimizer: disabled (Remix default)

Constructor arguments:

admin = 0xA072b15486bee1E56A9BeE1C6F627dcF8e21332

initialTransferAgent =

0xA072b15486bee1E56A9BeE1C6F627dcF8e21332

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0(0.0%)
High	0(0.0%)
Medium	0(0.0%)
Low	0(0.0%)
Informational	1 (100.0%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	0	1



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Use of Floating Pragma	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Using throw

✓ Using inline assembly

✓ Style guide violation

✓ Unsafe type inference

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Informational Issues

Use of Floating Pragma

Resolved

Path

TMREMinimalCustody.sol

Description

The contract specifies the Solidity compiler version using a floating pragma directive:

```
solidity  
>> pragma solidity ^0.8.20;
```

The caret (^) symbol allows the contract to be compiled with any compiler version from 0.8.20 up to, but not including, 0.9.0. This means the contract is not pinned to a single, deterministic compiler version.

Impact

Different compiler versions may introduce subtle behavioral differences, optimizations, or new bug fixes/introductions between minor versions. If the contract is compiled with a newer, untested compiler version in the future, it may produce unexpected bytecode or behavior that was not audited or tested against. This inconsistency between development, testing, and production environments can silently undermine security assumptions.

Likelihood

Low

Recommendation

Lock the pragma to a specific, well-tested compiler version that has been used during development



Functional Tests

Some of the tests performed are mentioned below:

TRIGGERMINT_AI_CAPITAL_REAL_ESTATE.sol

- ✓ Verified only DEFAULT_ADMIN_ROLE can whitelist or remove addresses.
- ✓ Verified only DEFAULT_ADMIN_ROLE can enable trading.
- ✓ Verified transfers revert before trading is enabled when the caller is not the Transfer Agent.
- ✓ Verified transfers succeed before trading is enabled when initiated by the Transfer Agent.
- ✓ Verified transfers revert in custodialMode when the caller is not the Transfer Agent.
- ✓ Verified transfers succeed in custodialMode when initiated by the Transfer Agent.
- ✓ Verified both sender and receiver must be whitelisted for transfers.
- ✓ Verified transfers revert if either sender or receiver is not whitelisted.
- ✓ Verified only admin can update the Transfer Agent.
- ✓ Verified new Transfer Agent receives the required role and whitelist status.
- ✓ Verified previous Transfer Agent role is revoked upon update.
- ✓ Verified Transfer Agent can burn tokens via surrenderForRedemption.
- ✓ Verified non-Transfer Agent cannot execute token burns.
- ✓ Verified total supply is minted to the initial Transfer Agent at deployment.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20.sol (OpenZeppelin v5)	Library / Base Contract	Core token logic – balances, transfers, allowances, _update() hook	Correct implementation of ERC-20 invariants; no hidden backdoors in inherited logic	Any vulnerability in OZ ERC-20 v5 propagates directly into TMRE. The overridden _update() silently interacts with every mint, burn, and transfer
AccessControl.sol (OpenZeppelin v5)	Library / Access Management	Role-based permission system for DEFAULT_ADMIN_ROLE and TRANSFER_AGENT_ROLE	grantRole, revokeRole, renounceRole behave exactly as documented; no undocumented role escalation paths	DEFAULT_ADMIN_ROLE is its own role admin by default – admin can self-escalate to any role. renounceRole can permanently orphan the contract. These are documented OZ warnings left unmitigated



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
solidity ^0.8.20 (Floating Pragma)	Compiler Tool	Compilation of all contract logic	Compiler produces deterministic, bug-free bytecode	Floating pragma allows compilation under untested minor versions (e.g., 0.8.28). Different bytecode may be produced across environments. Audited version may differ from deployed version

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
TMREMinimalCustody.sol	constructor()	What this function can do Mint 750,000 TMRE to initialTransferAgent. Grant DEFAULT_ADMIN_ROLE to admin. Grant TRANSFER_AGENT_ROLE to agent. Whitelist both addresses. Set custodialMode = true What this function cannot/should not do Mint again after deployment. Accept zero addresses. Deploy with admin == initialTransferAgent (not guarded) Main invariant(s) Total supply is permanently fixed at 750,000 TMRE at deployment Access level Implicit restriction executes once at deployment only



Contract Name	Function	Category
TMREMinimalCustody.sol	setWhitelisted()	What this function can do Add or remove any address from the transfer whitelist. Can remove an investor who still holds tokens, permanently freezing their balance with zero on-chain recovery What this function cannot/should not do Should not remove an address with a non-zero balance without a prior sweep or redemption mechanism Main invariant(s) Only whitelisted addresses may appear as from or to in any transfer. Mint and burn bypass this check Access level DEFAULT_ADMIN_ROLE only
TMREMinimalCustody.sol	enableTrading()	What this function can do Permanently flip tradingEnabled = true. Emit TradingEnabled event with LISTING_DATE_ID and current block.timestamp What this function cannot/should not do Cannot be reversed – no disableTrading() exists. LISTING_DATE_ID = 20260218 is decorative and imposes no real on-chain date restriction Main invariant(s) tradingEnabled is a permanent one-way latch – can never return to false Access level DEFAULT_ADMIN_ROLE only

Contract Name	Function	Category
TMREMinimalCustody.sol	setCustodialMode()	What this function can do Toggle whether only the Transfer Agent can initiate transfers, or any whitelisted address can freely transfer peer-to-peer What this function cannot/should not do Should not be disabled before secondary market compliance is established. Disabling does not retroactively cancel in-flight approve() allowances Main invariant(s) When custodialMode = true, msg.sender must hold TRANSFER_AGENT_ROLE on every non-mint/burn call Access level DEFAULT_ADMIN_ROLE only
TMREMinimalCustody.sol	setTransferAgent()	What this function can do TRANSFER_AGENT_ROLE to new agent. Revoke role from old agent. Update transferAgent pointer. Auto-whitelist new agent if not already listed What this function cannot/should not do Cannot force-transfer tokens held by the old agent. Does not prevent admin bypassing this via direct inherited grantRole(), leaving transferAgent pointer stale and misleading Main invariant(s) Exactly one address should hold TRANSFER_AGENT_ROLE at all times. transferAgent state variable must always reflect the true active role holder Access level DEFAULT_ADMIN_ROLE only

Contract Name	Function	Category
TMREMinimalCustody.sol	surrenderForRedemption()	What this function can do Burn any amount of tokens from any holder address without requiring that holder's on-chain approval, signature, or consent. Accept an empty referenced string with zero validation What this function cannot/should not do Should not burn tokens without verifiable holder consent. Should not accept an empty referenced – permanently breaks the off-chain audit trail for that burn event Main invariant(s) Burned tokens reduce totalSupply() permanently. TOTAL_SUPPLY constant remains 750,000 regardless, misleading all integrators post-burn Access level TRANSFER_AGENT_ROLE only

Contract Name	Function	Category
TMREMinimalCustody.sol	_update() (internal override)	What this function can do Enforce whitelist on both from and to. Block non-agent transfers pre-listing. Block non-agent transfers in custodial mode. Fires on every mint, transfer, and burn in the contract What this function cannot/should not do Does not distinguish transfer() from transferFrom() – msg.sender in transferFrom is the spender, silently breaking standard ERC-20 composability. Contains redundant dual hasRole check when both ! tradingEnabled and custodialMode are simultaneously true Main invariant(s) Mint (from = address(0)) and burn (to = address(0)) bypass all whitelist and role checks entirely Access level Internal – automatically triggered by all ERC-20 movement functions

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets	Centralization Risk
TMREMinimalCustody.sol (Transfer Agent wallet)	ERC-20 (TMRE)	Full initial supply of 750,000 TMRE minted directly to Transfer Agent at deployment	100% of supply under single EOA/key at genesis
Investor wallets (whitelisted)	ERC-20 (TMRE)	Distributed investor holdings post-distribution	Holdings are frozen the moment admin calls <code>setWhitelisted(investor, false)</code>

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TMREMinimalCustody	constructor()	–	750,000 TMRE minted to Transfer Agent	Deployer (once)	Single-point genesis risk. All supply under one key immediately
TMREMinimalCustody	Transfer Agent → Investor (transfer)	–	TMRE to investor	Transfer Agent only (custodial mode)	Pre-listing distribution phase. No investor consent model exists on-chain
TMREMinimalCustody	Investor → Investor (transfer)	TMRE from sender	TMRE to receiver	Whitelisted address (if custodial off)	Only possible after setCustodialMode(false). Both parties must be whitelisted
TMREMinimalCustody	surrenderForRedemption()	TMRE burned from holder	– (destroyed)	Transfer Agent only	Most dangerous exit point. No holder approval required. Empty referenceld breaks audit trail. Irreversible
TMREMinimalCustody	setWhitelisted(addr, false)	–	– (no token movement)	Admin only	Soft exit no tokens move but holder is effectively frozen. Precondition to the burn rug vector

Closing Summary

In this report, we have considered the security of TRIGGERMINT AI CAPITAL. We performed our audit according to the procedure described above.

No critical issues in TRIGGERMINT AI CAPITAL token. One issue of Informational severity was found, the TRIGGERMINT AI CAPITAL team resolved the mentioned issue.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

February 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com